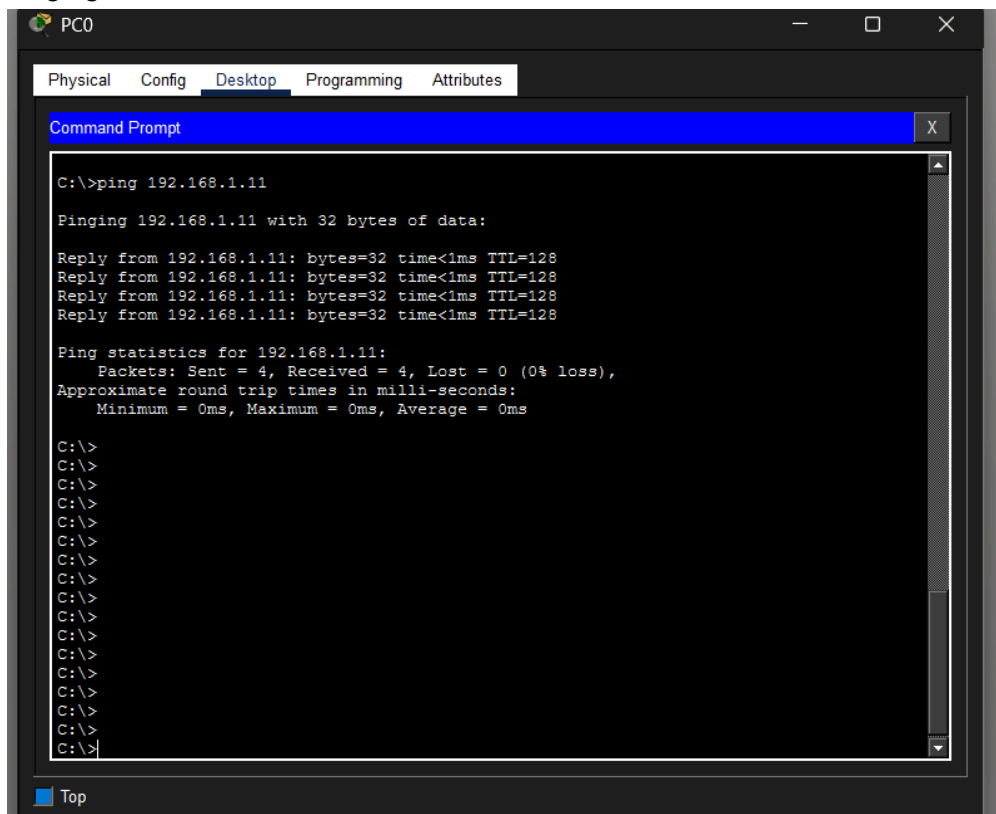
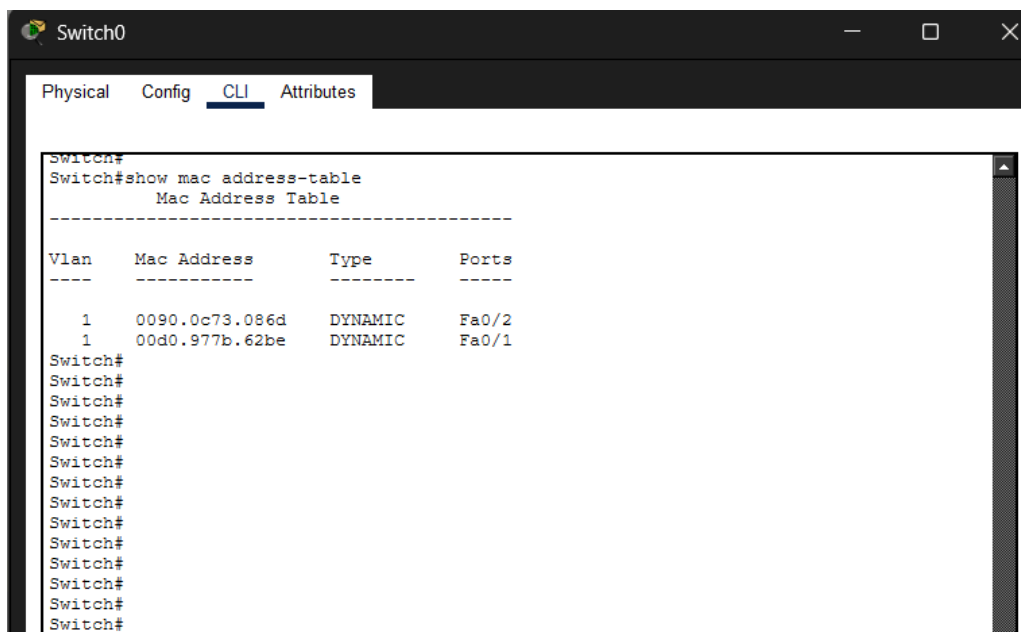


Pinging PC1 from PC0



The switch learns the mac addresses



Ping PC5 from PC0

```
C:\>
C:\>
C:\>
C:\>
C:\>ping 192.168.1.15

Pinging 192.168.1.15 with 32 bytes of data:

Reply from 192.168.1.15: bytes=32 time<1ms TTL=128
Reply from 192.168.1.15: bytes=32 time<1ms TTL=128
Reply from 192.168.1.15: bytes=32 time<1ms TTL=128
Reply from 192.168.1.15: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.15:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>
```

[Top](#)

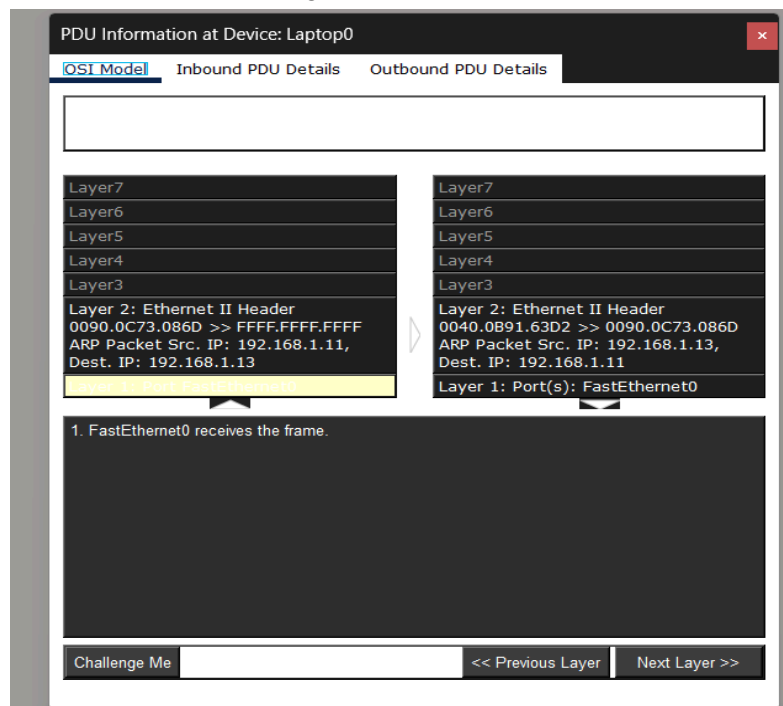
The mac address table gets updated

```
Switch#
Switch#show mac address-table
      Mac Address Table
-----
Vlan  Mac Address      Type      Ports
----  -
1     0060.7061.adcc    DYNAMIC   Fa0/6
1     0090.0c73.086d    DYNAMIC   Fa0/2
1     00d0.977b.62be    DYNAMIC   Fa0/1
Switch#
```

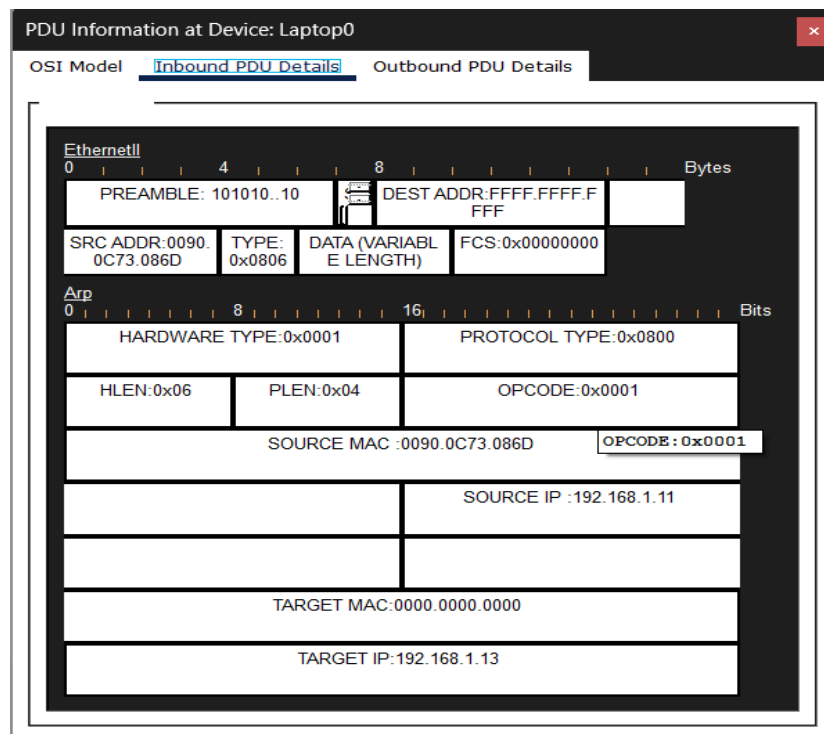
[Copy](#) [Paste](#)

ARP Request and Reply

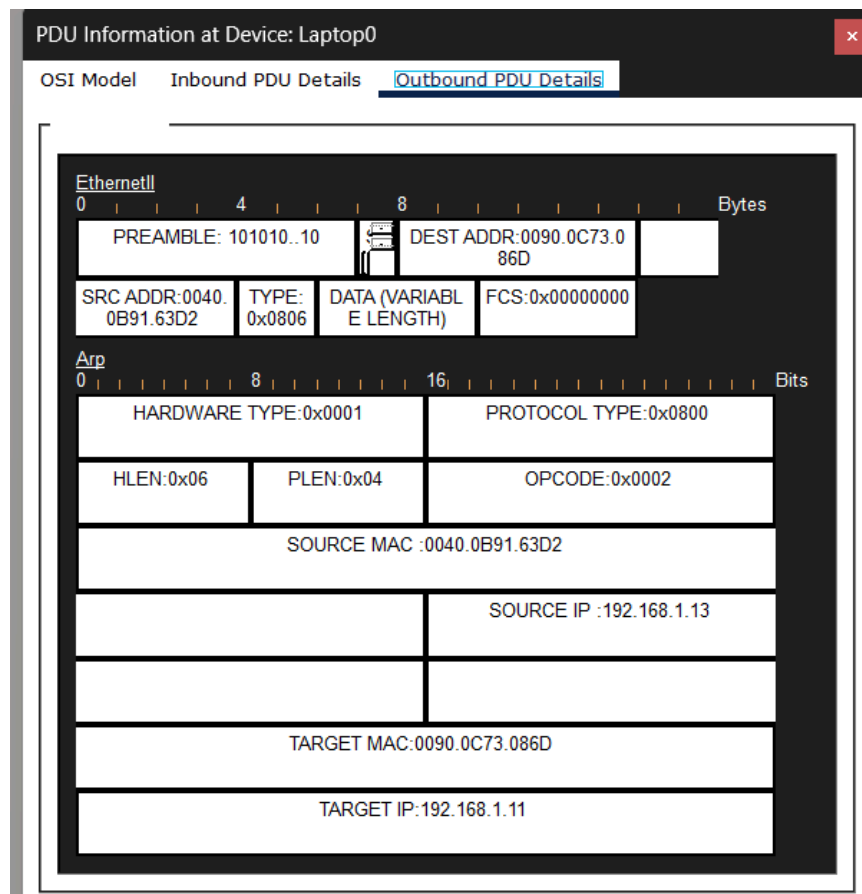
Since the frame is being broadcasted the destination MAC address is FFFF.FFFF.FFFF



The target mac is 0000.0000.0000 because the pc is asking for any system which has the IP of 192.168.1.13 to provide its mac address



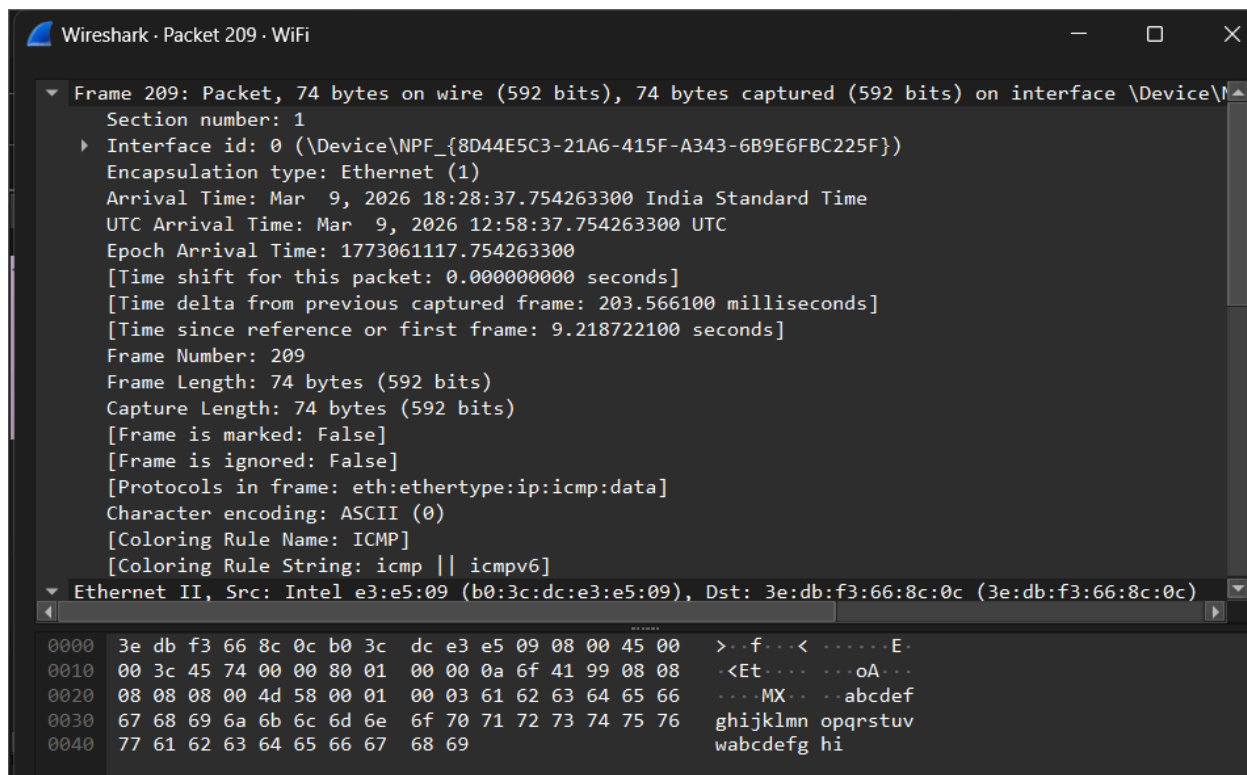
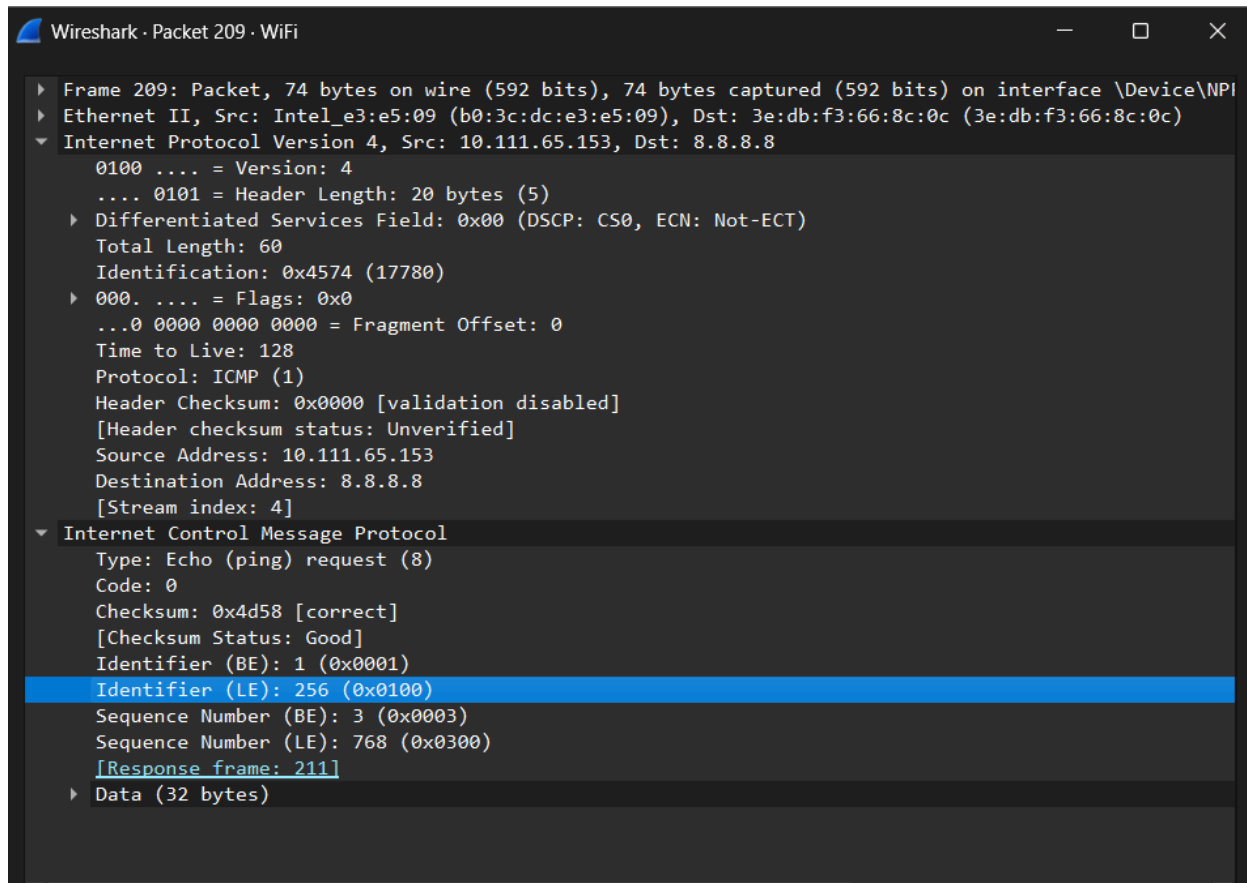
The laptop0 sends its mac address to the destination pc which requested the address in the first place.



2. Capture and analyze Ethernet frames using Wireshark. Inspect the structure of the frame, including destination and source MAC addresses, Ethertype, payload, and FCS. Use GNS3 or Packet Tracer to simulate network traffic.

The packets are captured while pinging 8.8.8.8

Capturing from WiFi						
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
icmp						
No.	Time	Source	Destination	Protocol	Length	Info
209	9.218722100	10.111.65.153	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=3/768, ttl=128 (reply in 211)
211	9.262886800	8.8.8.8	10.111.65.153	ICMP	74	Echo (ping) reply id=0x0001, seq=3/768, ttl=113 (request in 209)
218	10.231787900	10.111.65.153	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=4/1024, ttl=128 (reply in 219)
219	10.272554100	8.8.8.8	10.111.65.153	ICMP	74	Echo (ping) reply id=0x0001, seq=4/1024, ttl=113 (request in 218)
220	11.246266500	10.111.65.153	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=5/1280, ttl=128 (reply in 221)
221	11.273605900	8.8.8.8	10.111.65.153	ICMP	74	Echo (ping) reply id=0x0001, seq=5/1280, ttl=113 (request in 220)
223	12.265548000	10.111.65.153	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=6/1536, ttl=128 (reply in 224)
224	12.306168000	8.8.8.8	10.111.65.153	ICMP	74	Echo (ping) reply id=0x0001, seq=6/1536, ttl=113 (request in 223)



- The frame contains an Ethernet II header with source and destination MAC addresses.
- The Ethertype field indicates that the payload contains an IPv4 packet.
- The IPv4 packet includes source and destination IP addresses along with a TTL value used to limit packet lifetime in the network.
- The IPv4 payload carries an ICMP Echo Request packet which was generated by the ping command.
- The ICMP header contains fields such as type, code, identifier, and sequence number used to manage echo requests and replies.
- The frame also includes a Frame Check Sequence for error detection at the data link layer but this isn't displayed by Wireshark because it's stripped away by the NIC before the packet reaches the OS.

3. Configure static IP addresses, modify MAC addresses, and verify network connectivity using ping and ifconfig commands.

```
yasha@yasha-VirtualBox: ~
yasha@yasha-VirtualBox:~$ sudo ip addr flush dev enp0s8
yasha@yasha-VirtualBox:~$ ip link show enp0s8
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFA
ULT group default qlen 1000
    link/ether 08:00:27:fe:9f:8e brd ff:ff:ff:ff:ff:ff
```

Statically assigning an IP to the interface

```
yasha@yasha-VirtualBox:~$ sudo ip addr add 192.168.56.10/24 dev enp0s8
yasha@yasha-VirtualBox:~$ sudo ip link set enp0s8 up
yasha@yasha-VirtualBox:~$ ip addr show enp0s8
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group def
ault qlen 1000
    link/ether 08:00:27:fe:9f:8e brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s8
        valid_lft 240sec preferred_lft 240sec
    inet 192.168.56.10/24 scope global secondary enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a2a2:6e80:9b0d:3784/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
yasha@yasha-VirtualBox:~$
```

Communicating between windows and linux vm using the VirtualBox host only adapter

```
C:\Users\yasha>ping 192.168.56.10

Pinging 192.168.56.10 with 32 bytes of data:
Request timed out.
Reply from 192.168.56.10: bytes=32 time=1ms TTL=64
Reply from 192.168.56.10: bytes=32 time=1ms TTL=64
Reply from 192.168.56.10: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.56.10:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 1ms, Average = 1ms

C:\Users\yasha>ping 192.168.56.10

Pinging 192.168.56.10 with 32 bytes of data:
Reply from 192.168.56.10: bytes=32 time=2ms TTL=64
Reply from 192.168.56.10: bytes=32 time=2ms TTL=64
Reply from 192.168.56.10: bytes=32 time=2ms TTL=64
Reply from 192.168.56.10: bytes=32 time=4ms TTL=64

Ping statistics for 192.168.56.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2ms, Maximum = 4ms, Average = 2ms

C:\Users\yasha>
```

```
yasha@yasha-VirtualBox:~$ ping -c 3 192.168.56.1
PING 192.168.56.1 (192.168.56.1) 56(84) bytes of data.
64 bytes from 192.168.56.1: icmp_seq=1 ttl=255 time=3.22 ms
64 bytes from 192.168.56.1: icmp_seq=2 ttl=255 time=4.15 ms
64 bytes from 192.168.56.1: icmp_seq=3 ttl=255 time=4.03 ms

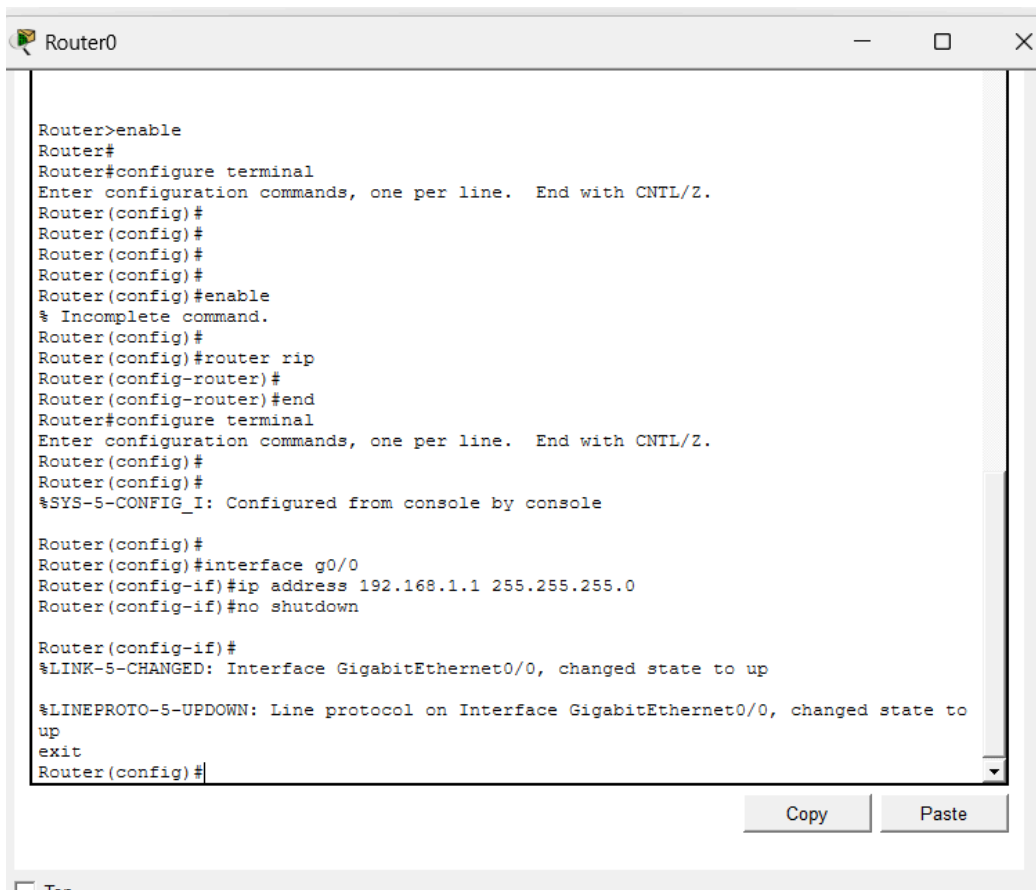
--- 192.168.56.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 3.222/3.800/4.146/0.411 ms
```

- To configure a static IP address on the Linux virtual machine use `ip addr add` command.
- The interface `enp0s8` belongs to the VirtualBox host-only network and normally receives the IP address `192.168.56.101` dynamically from the DHCP server provided by the host.
- When the command `sudo ip addr add 192.168.56.10/24 dev enp0s8` is executed, a secondary static IP address is assigned to the interface while the original DHCP address still remains.
- The command `ip addr flush dev enp0s8` removes all IP addresses from that interface.

- If the default DHCP configuration is required again, the manually added address can be removed using `sudo ip addr del 192.168.56.10/24 dev enp0s` or the network service can be restarted so that the DHCP client reassigns the default address.
- Connectivity between the host and the VM can then be verified using the `ping` command.

4. Troubleshoot Ethernet Communication with ping and traceroute -> Using cisco packet tracer.

Setting up the default gateway for router0



```

Router>enable
Router#
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#
Router(config)#
Router(config)#
Router(config)#
Router(config)#enable
% Incomplete command.
Router(config)#
Router(config)#router rip
Router(config-router)#
Router(config-router)#end
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#
Router(config)#
%SYS-5-CONFIG_I: Configured from console by console

Router(config)#
Router(config)#interface g0/0
Router(config-if)#ip address 192.168.1.1 255.255.255.0
Router(config-if)#no shutdown

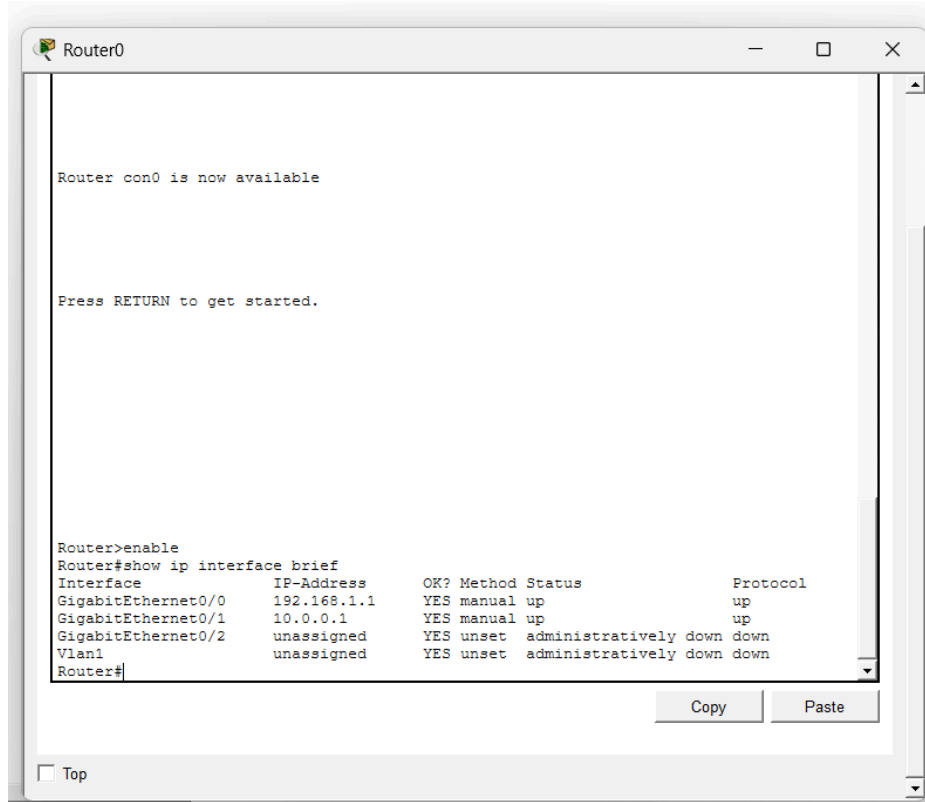
Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to
up
exit
Router(config)#

```

Copy Paste

Interfaces of router0



Router con0 is now available

Press RETURN to get started.

```
Router>enable
Router#show ip interface brief
```

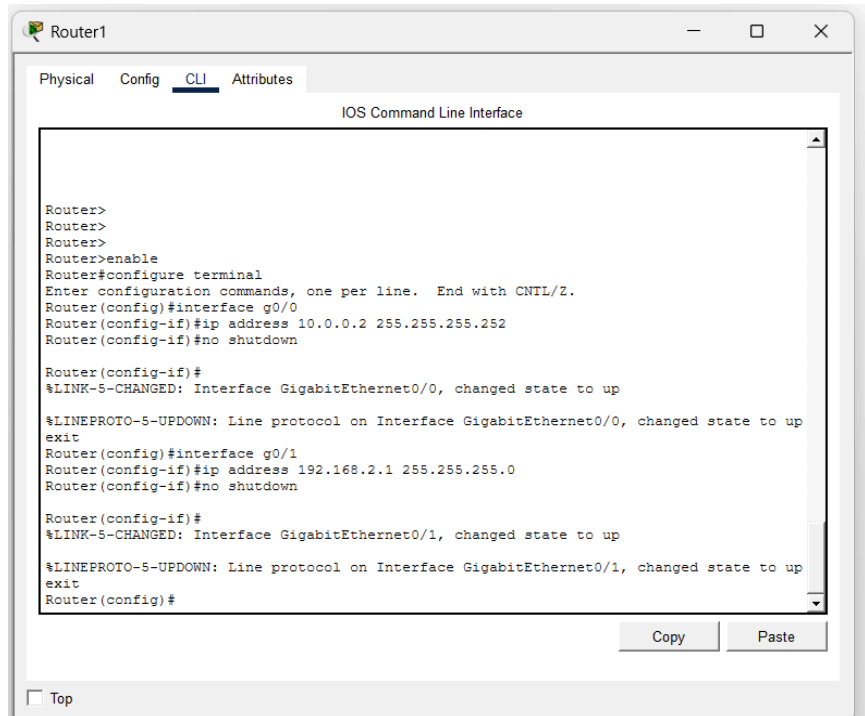
Interface	IP-Address	OK?	Method	Status	Protocol
GigabitEthernet0/0	192.168.1.1	YES	manual	up	up
GigabitEthernet0/1	10.0.0.1	YES	manual	up	up
GigabitEthernet0/2	unassigned	YES	unset	administratively down	down
Vlan1	unassigned	YES	unset	administratively down	down

Router#

Copy Paste

☐ Top

Setting up the default gateway for router1



Physical Config CLI Attributes

IOS Command Line Interface

```
Router>
Router>
Router>
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface g0/0
Router(config-if)#ip address 10.0.0.2 255.255.255.252
Router(config-if)#no shutdown

Router(config-if)#
$LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up
$LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up
exit
Router(config)#interface g0/1
Router(config-if)#ip address 192.168.2.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
$LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up
$LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up
exit
Router(config)#
```

Copy Paste

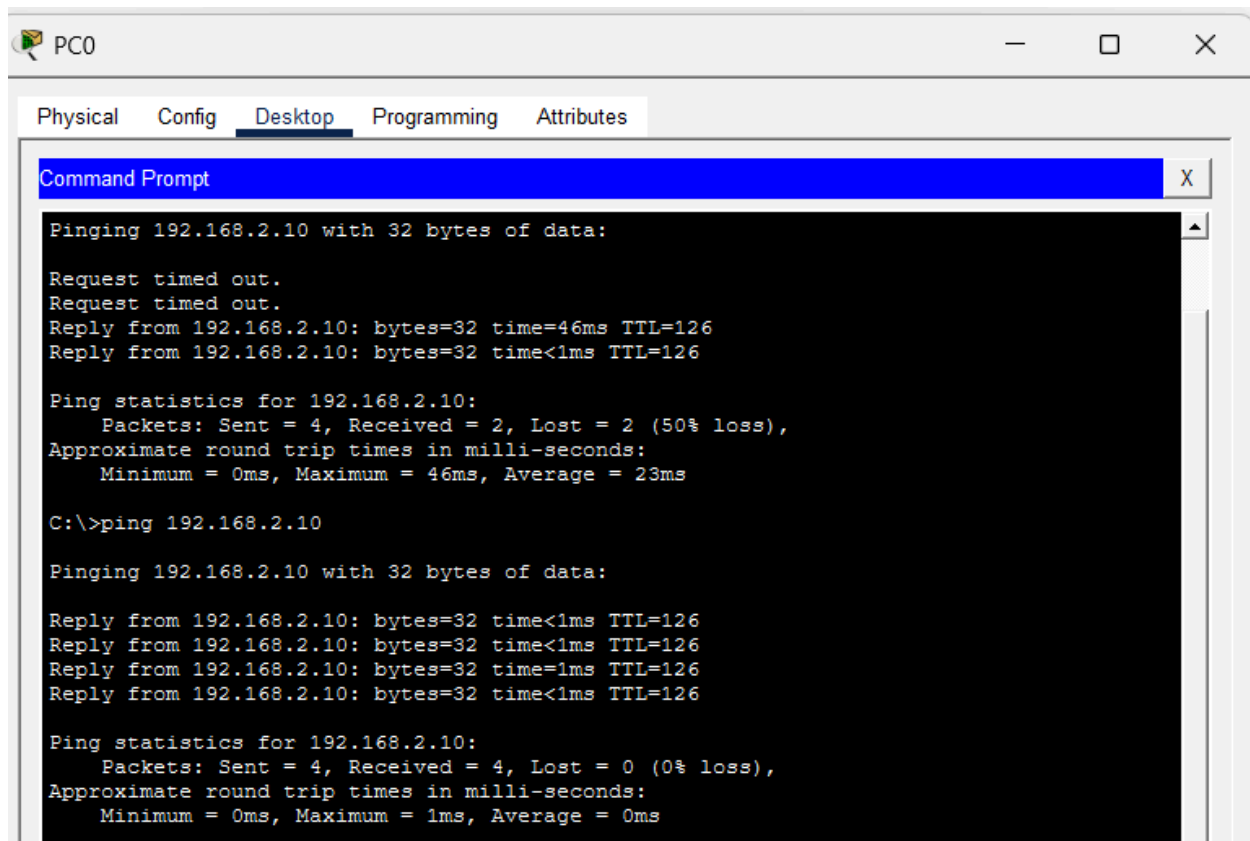
☐ Top

Setting the static routes for Router0 and Router1

```
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 192.168.2.0 255.255.255.0 10.0.0.2
Router(config)#
```

```
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 192.168.1.0 255.255.255.0 10.0.0.1
Router(config)#
```

Pinging pc1



The screenshot shows a window titled "PC0" with tabs for Physical, Config, Desktop, Programming, and Attributes. The "Desktop" tab is active, displaying a "Command Prompt" window. The Command Prompt shows the results of a ping command to 192.168.2.10. The first attempt shows a 50% loss of packets, while the second attempt shows 0% loss.

```
Command Prompt
Pinging 192.168.2.10 with 32 bytes of data:

Request timed out.
Request timed out.
Reply from 192.168.2.10: bytes=32 time=46ms TTL=126
Reply from 192.168.2.10: bytes=32 time<1ms TTL=126

Ping statistics for 192.168.2.10:
    Packets: Sent = 4, Received = 2, Lost = 2 (50% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 46ms, Average = 23ms

C:\>ping 192.168.2.10

Pinging 192.168.2.10 with 32 bytes of data:

Reply from 192.168.2.10: bytes=32 time<1ms TTL=126
Reply from 192.168.2.10: bytes=32 time<1ms TTL=126
Reply from 192.168.2.10: bytes=32 time=1ms TTL=126
Reply from 192.168.2.10: bytes=32 time<1ms TTL=126

Ping statistics for 192.168.2.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

Tracing the route taken

```
C:\>tracert 192.168.2.10

Tracing route to 192.168.2.10 over a maximum of 30 hops:

  1  0 ms    0 ms    0 ms    192.168.1.1
  2  0 ms    1 ms    0 ms    10.0.0.2
  3  0 ms    0 ms    0 ms    192.168.2.10

Trace complete.

C:\>
```

Removing the route

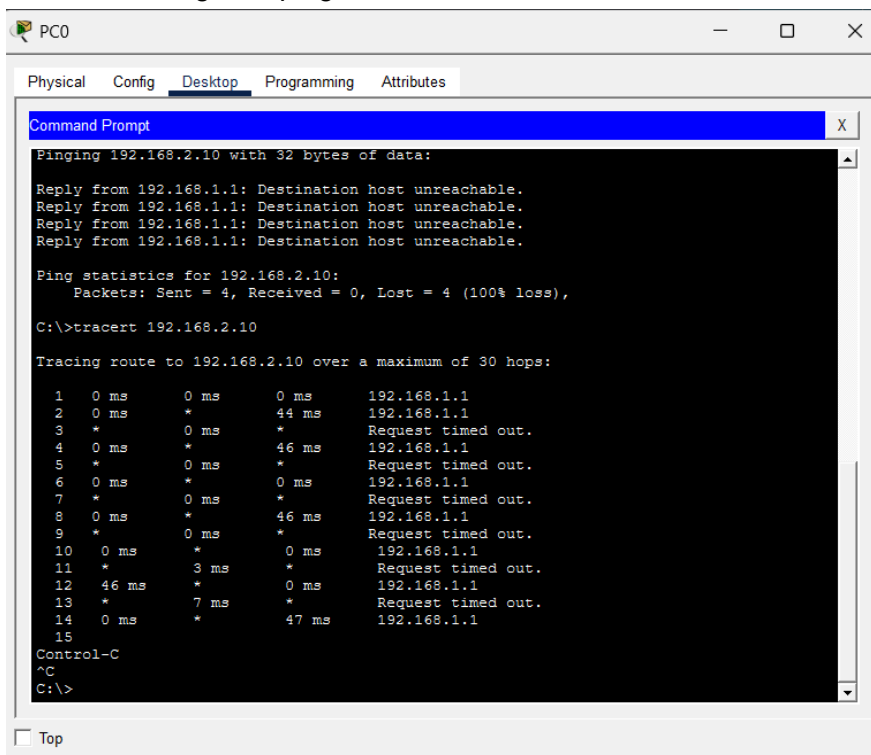
```
Router#show ip interface brief
Interface                IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0       192.168.1.1     YES manual  up          up
GigabitEthernet0/1       10.0.0.1        YES manual  up          up
GigabitEthernet0/2       unassigned      YES unset   administratively down down
Vlan1                    unassigned      YES unset   administratively down down
Router#
Router#
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#no ip route 192.168.2.0 255.255.255.0 10.0.0.2
Router(config)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console
```

Copy

Paste

☐ Top

Troubleshooting with ping and tracer



The screenshot shows a window titled 'PC0' with tabs for 'Physical', 'Config', 'Desktop', 'Programming', and 'Attributes'. The 'Desktop' tab is active, displaying a 'Command Prompt' window. The Command Prompt shows the following output:

```
Pinging 192.168.2.10 with 32 bytes of data:
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.

Ping statistics for 192.168.2.10:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>tracert 192.168.2.10

Tracing route to 192.168.2.10 over a maximum of 30 hops:

  0  0 ms    0 ms    0 ms    192.168.1.1
  1  0 ms    *        44 ms   192.168.1.1
  2  *        0 ms    *        Request timed out.
  3  0 ms    *        46 ms   192.168.1.1
  4  *        0 ms    *        Request timed out.
  5  0 ms    *        0 ms    192.168.1.1
  6  *        0 ms    *        Request timed out.
  7  0 ms    *        46 ms   192.168.1.1
  8  *        0 ms    *        Request timed out.
  9  0 ms    *        0 ms    192.168.1.1
 10  *        3 ms    *        Request timed out.
 11  46 ms    *        0 ms    192.168.1.1
 12  *        7 ms    *        Request timed out.
 13  0 ms    *        47 ms   192.168.1.1
 14
 15
Control-C
^C
C:\>
```

- Ethernet communication between two hosts can be tested and troubleshoot using ping and traceroute commands in Cisco Packet Tracer.
- Initially two PCs are connected through switches and routers with proper IP addressing and routing configuration.
- The ping command is used to verify connectivity between the source and destination hosts by sending ICMP Echo Request packets and receiving Echo Replies.
- The traceroute command was executed to identify the path taken by packets across intermediate routers. Each hop in the network is displayed allowing the detection of routing issues or network failures.
- By intentionally removing routing entries, communication failures can be observed and diagnosed using traceroute.

5. Create a simple LAN setup with two Linux machines connected via a switch.

- A simple LAN with two Linux machines can be created either using a physical Ethernet switch or a virtual switch.
- When using virtualization software such as VirtualBox, a physical switch is not required because the hypervisor provides a virtual networking interface that functions like a switch. Both Linux virtual machines can be connected to the same virtual network (Host-Only or Internal Network).
- Each machine is then assigned an IP address in the same subnet, allowing them to communicate.

On LinuxVM2:

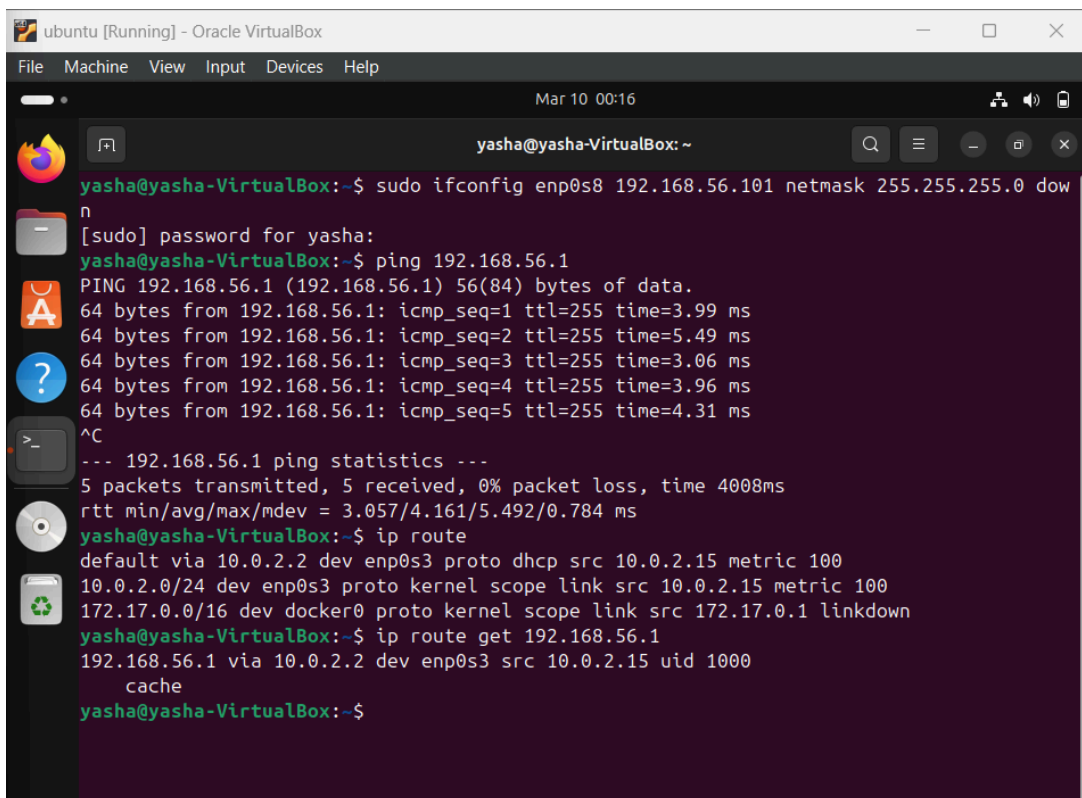
- `sudo ip addr add 192.168.50.10/24 dev enp0s8`
- `sudo ip link set enp0s8 up`

On LinuxVM2:

- `sudo ip addr add 192.168.50.11/24 dev enp0s8`
- `sudo ip link set enp0s8 up`

The virtualization platform provides a virtual switch, so a physical switch is not required.

6. Ping from one machine to the other. If it fails, use `ifconfig` to ensure the IP addresses are configured correctly.



```
ubuntu [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Mar 10 00:16
yasha@yasha-VirtualBox: ~
yasha@yasha-VirtualBox:~$ sudo ifconfig enp0s8 192.168.56.101 netmask 255.255.255.0 down
[sudo] password for yasha:
yasha@yasha-VirtualBox:~$ ping 192.168.56.1
PING 192.168.56.1 (192.168.56.1) 56(84) bytes of data:
64 bytes from 192.168.56.1: icmp_seq=1 ttl=255 time=3.99 ms
64 bytes from 192.168.56.1: icmp_seq=2 ttl=255 time=5.49 ms
64 bytes from 192.168.56.1: icmp_seq=3 ttl=255 time=3.06 ms
64 bytes from 192.168.56.1: icmp_seq=4 ttl=255 time=3.96 ms
64 bytes from 192.168.56.1: icmp_seq=5 ttl=255 time=4.31 ms
^C
--- 192.168.56.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4008ms
rtt min/avg/max/mdev = 3.057/4.161/5.492/0.784 ms
yasha@yasha-VirtualBox:~$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
yasha@yasha-VirtualBox:~$ ip route get 192.168.56.1
192.168.56.1 via 10.0.2.2 dev enp0s3 src 10.0.2.15 uid 1000
cache
yasha@yasha-VirtualBox:~$
```

```

C:\Users\yasha>
C:\Users\yasha>
C:\Users\yasha>
C:\Users\yasha>ping 192.168.56.101

Pinging 192.168.56.101 with 32 bytes of data:
Request timed out.
Request timed out.
Reply from 192.168.56.1: Destination host unreachable.
Request timed out.

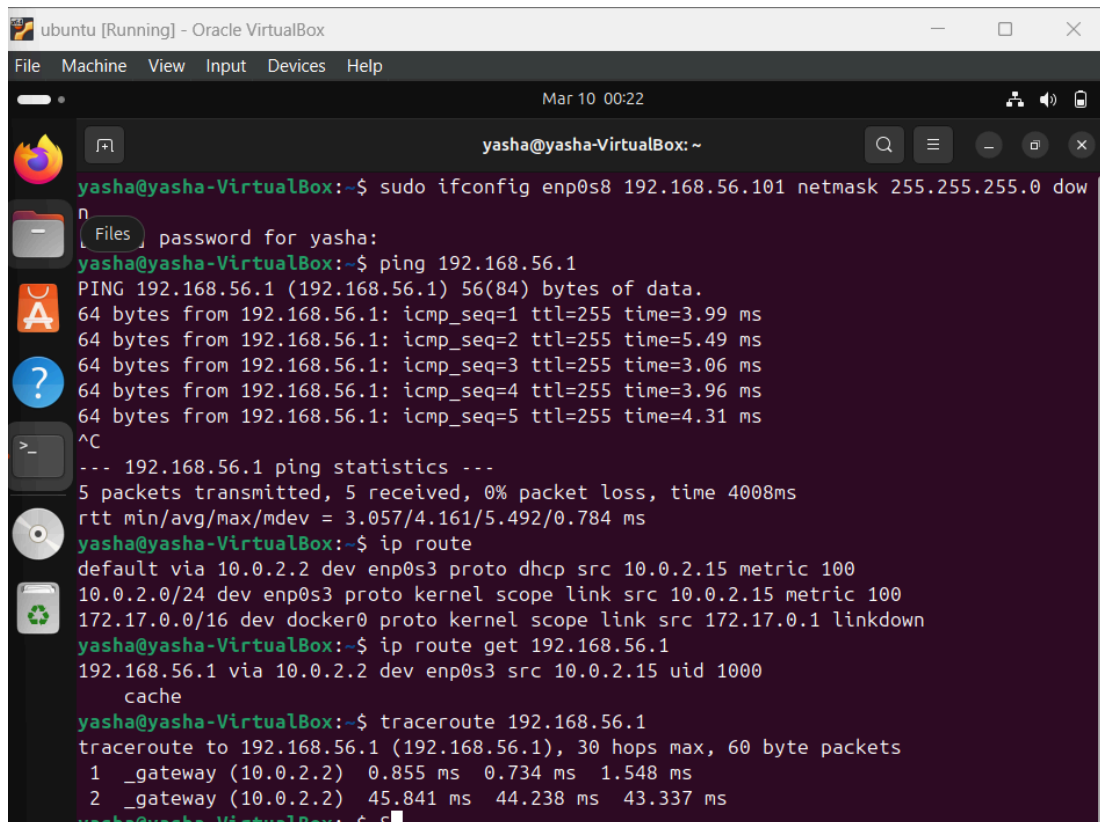
Ping statistics for 192.168.56.101:
    Packets: Sent = 4, Received = 1, Lost = 3 (75% loss),

C:\Users\yasha>

```

- When the Linux interface `enp0s8` is brought down the route to the `192.168.56.0/24` network is removed from the routing table.
- As a result, Linux used the default route through the NAT interface `enp0s3` to reach the destination `192.168.56.1`, which allowed the ping to succeed.
- However, when the Windows machine attempted to ping the Linux machine, the request failed because the Linux interface responsible for that network was down and could not respond to ARP requests.

7. Use traceroute to identify where the packets are being dropped if the ping fails.



```

ubuntu [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Mar 10 00:22
yasha@yasha-VirtualBox: ~
yasha@yasha-VirtualBox:~$ sudo ifconfig enp0s8 192.168.56.101 netmask 255.255.255.0 down
n
Files password for yasha:
yasha@yasha-VirtualBox:~$ ping 192.168.56.1
PING 192.168.56.1 (192.168.56.1) 56(84) bytes of data.
64 bytes from 192.168.56.1: icmp_seq=1 ttl=255 time=3.99 ms
64 bytes from 192.168.56.1: icmp_seq=2 ttl=255 time=5.49 ms
64 bytes from 192.168.56.1: icmp_seq=3 ttl=255 time=3.06 ms
64 bytes from 192.168.56.1: icmp_seq=4 ttl=255 time=3.96 ms
64 bytes from 192.168.56.1: icmp_seq=5 ttl=255 time=4.31 ms
^C
--- 192.168.56.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4008ms
rtt min/avg/max/mdev = 3.057/4.161/5.492/0.784 ms
yasha@yasha-VirtualBox:~$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
yasha@yasha-VirtualBox:~$ ip route get 192.168.56.1
192.168.56.1 via 10.0.2.2 dev enp0s3 src 10.0.2.15 uid 1000
cache
yasha@yasha-VirtualBox:~$ traceroute 192.168.56.1
traceroute to 192.168.56.1 (192.168.56.1), 30 hops max, 60 byte packets
 1 _gateway (10.0.2.2) 0.855 ms 0.734 ms 1.548 ms
 2 _gateway (10.0.2.2) 45.841 ms 44.238 ms 43.337 ms
yasha@yasha-VirtualBox:~$ S

```

```

C:\Users\yasha>tracert 192.168.56.101

Tracing route to 192.168.56.101 over a maximum of 30 hops

  1     3 ms      7 ms      3 ms    10.240.251.98
  2     *         *         *        Request timed out.
  3     *         *         *        Request timed out.
  4     *         *         *        Request timed out.
  5    61 ms     23 ms     30 ms    172.27.251.99
  6     *         *         *        Request timed out.
  7     *         *         *        Request timed out.
  8     *         *         *        Request timed out.
  9     *         *         *        Request timed out.
 10     *         *         *        Request timed out.
 11     *         *         *        Request timed out.
 12     *         *         *        Request timed out.
 13     *         *         *        Request timed out.
 14     *         *         *        Request timed out.
 15     *         *         *        Request timed out.
 16     *         *         *        Request timed out.
 17     *         *         *        Request timed out.
 18     *         *         *        Request timed out.
 19     *         *         *        Request timed out.
 20     *         *         *        Request timed out.
 21     *         *         *        Request timed out.
 22     *         *         *        Request timed out.
 23     *         *         *        Request timed out.
 24     *         *         *        Request timed out.
 25     *         *         *        Request timed out.
 26     *         *         *        Request timed out.
 27     *         *         *        Request timed out.
 28     *         *         *        Request timed out.
 29     *         *         *        Request timed out.
 30     *         *         *        Request timed out.

Trace complete.

C:\Users\yasha>

```

If the ping command fails, the traceroute tool can be used to determine where packets are being dropped in the network. The command `traceroute destination_IP` in Linux or `tracert destination_IP` in Windows shows the path that packets take through intermediate devices. Each hop represents a router or network device. If the output shows * symbols after a certain hop, it indicates that the packets are being dropped at that point in the network. This helps identify where the connectivity issue is occurring.

8. Research the Linux kernel's handling of Ethernet devices and network interfaces. Write a short report on how the Linux kernel supports Ethernet communication (referencing kernel.org documentation).

The Linux kernel provides builtin support for Ethernet communication through its networking subsystem and device driver framework. Ethernet interfaces in Linux are managed as network devices that interact with the kernel's networking stack to transmit and receive packets. This architecture allows the operating system to communicate with Ethernet hardware while providing a consistent interface for applications and networking utilities.

Ethernet Device Drivers

In Linux, Ethernet communication is primarily handled through Ethernet device drivers. These drivers allow the kernel to interact directly with hardware network interface cards (NICs). The Linux kernel contains many Ethernet drivers for different hardware vendors such as Intel, Marvell, and others. These drivers manage operations such as packet transmission, reception, and configuration of the Ethernet controller.

Device drivers translate generic networking operations from the kernel into hardware-specific commands for the NIC. They also handle interrupts generated when packets arrive and manage buffers used to store network data.

Network Device Interface

The kernel represents network interfaces using the network device abstraction (`net_device`). Each Ethernet interface (`eth0` or `enp0s3`) is registered as a network device in the kernel. This abstraction allows the networking stack to treat different network hardware in a uniform way. The kernel networking APIs provide functions for registering, configuring, and controlling these network devices.

Through this abstraction, system tools such as `ip`, `ifconfig`, and `ethtool` can configure Ethernet interfaces, set IP addresses, and manage link states.

Packet Processing in the Kernel

When an Ethernet packet arrives at the network interface card, the device driver passes the packet to the Linux networking stack. The kernel processes the packet through multiple layers including Ethernet, IP, and transport protocols. For outgoing packets, the process happens in reverse: data generated by applications is passed through the networking stack, encapsulated in Ethernet frames, and transmitted by the NIC.

This layered design allows Linux to support multiple network protocols while maintaining efficient communication between software and hardware.

PHY and Hardware Abstraction

The Linux kernel also includes a PHY abstraction layer that manages communication between the Ethernet controller and the physical network interface hardware. This layer handles tasks such as link negotiation, speed configuration, and hardware signaling over the network medium.

The Linux kernel supports Ethernet communication through a modular networking architecture that includes Ethernet device drivers, a network device abstraction layer, and a complete networking stack. These components allow the kernel to interact with different Ethernet hardware while providing a consistent interface for applications and system tools. Through this design, Linux can efficiently send and receive network packets across Ethernet networks.

9. Describe how you would configure a basic LAN interface using the `ip` command in Linux ([kernel.org](https://www.kernel.org)).

```
yasha@yasha-VirtualBox:~$ sudo ifconfig enp0s8 192.168.56.101 netmask 255.255.255.0 up
yasha@yasha-VirtualBox:~$ ip addr show enp0s8
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:fe:9f:8e brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global enp0s8
        valid_lft forever preferred_lft forever
yasha@yasha-VirtualBox:~$ sudo ip addr add 192.168.1.10/24 dev enp0s8
yasha@yasha-VirtualBox:~$ ip addr show enp0s8
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:fe:9f:8e brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet 192.168.1.10/24 scope global enp0s8
        valid_lft forever preferred_lft forever
yasha@yasha-VirtualBox:~$
```

In Linux, network interfaces can be configured using the `ip` command, which is part of the `iproute2` utilities used to manage networking in the Linux kernel. According to documentation from The Linux Kernel Organization, the kernel represents network interfaces as network devices that can be configured using user-space tools such as `ip`.

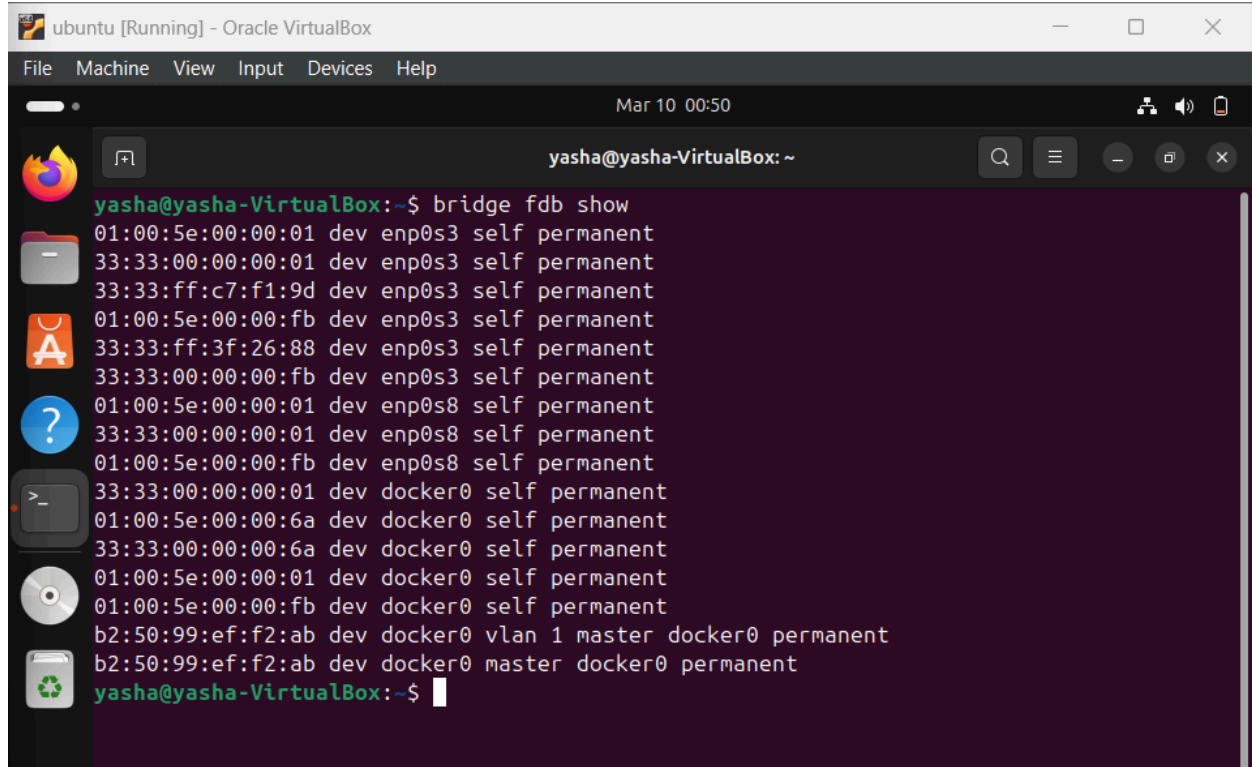
To configure a basic LAN interface, the first step is to check the available network interfaces. After assigning the address, the interface must be activated using `ip link set enp0s8 up`. This enables the interface so that it can send and receive packets on the LAN.

To verify the configuration, the following command can be used:

```
ip addr show enp0s8
```

This displays the interface status and the assigned IP address. Once configured, the system can communicate with other devices on the local network using tools such as ping.

10. Use Linux to view the MAC address table of a switch (if using a Linux-based network switch). Use the bridge or ip link commands to inspect the MAC table and demonstrate a basic switch's operation.



The screenshot shows a terminal window titled 'yasha@yasha-VirtualBox: ~' with a dark purple background. The command 'bridge fdb show' has been executed, displaying a list of MAC addresses and their associated interfaces. The output is as follows:

```
yasha@yasha-VirtualBox:~$ bridge fdb show
01:00:5e:00:00:01 dev enp0s3 self permanent
33:33:00:00:00:01 dev enp0s3 self permanent
33:33:ff:c7:f1:9d dev enp0s3 self permanent
01:00:5e:00:00:fb dev enp0s3 self permanent
33:33:ff:3f:26:88 dev enp0s3 self permanent
33:33:00:00:00:fb dev enp0s3 self permanent
01:00:5e:00:00:01 dev enp0s8 self permanent
33:33:00:00:00:01 dev enp0s8 self permanent
01:00:5e:00:00:fb dev enp0s8 self permanent
33:33:00:00:00:01 dev docker0 self permanent
01:00:5e:00:00:6a dev docker0 self permanent
33:33:00:00:00:6a dev docker0 self permanent
01:00:5e:00:00:01 dev docker0 self permanent
01:00:5e:00:00:fb dev docker0 self permanent
b2:50:99:ef:f2:ab dev docker0 vlan 1 master docker0 permanent
b2:50:99:ef:f2:ab dev docker0 master docker0 permanent
yasha@yasha-VirtualBox:~$
```

In Linux, a software-based Ethernet switch can be created and managed using the bridge subsystem of the Linux kernel. The bridge behaves like a Layer-2 switch and maintains a MAC address forwarding table, which maps MAC addresses to specific interfaces. This allows frames to be forwarded to the correct destination device.

To inspect the MAC address table of a Linux bridge, the bridge command can be used.

Display the MAC address forwarding database (FDB) using:

```
bridge fdb show
```

This command lists MAC addresses learned by the bridge along with the associated interface. Each entry shows which device the MAC address was learned from, demonstrating how the switch dynamically learns addresses from incoming Ethernet frames.

This indicates that the bridge `br0` has learned the MAC addresses of devices connected through interfaces. The Linux bridge uses this table to forward Ethernet frames only to the correct port instead of broadcasting them to all ports.

The `bridge fdb show` command allows us to inspect the MAC address table and observe how the Linux kernel implements basic Ethernet switching functionality.